

University of Glasgow

LAB 3

Real-Time Computing Systems & Architecture

Real Time Operating Systems (RTOS)

1. Introduction

A Real-Time Operating System, commonly abbreviated as RTOS, is a specialized software system designed to manage and execute tasks in real-time applications with strict timing requirements. Unlike general-purpose operating systems, like Windows, Linux or Mac OS, RTOS is engineered to ensure that critical tasks receive timely attention, making it ideal for applications where precise timing, reliability, and determinism are crucial. From embedded systems in automotive control units to aerospace and industrial automation, RTOS plays a vital role in enabling efficient and predictable operation in a wide range of real-time applications.

The STM IDE integrates free RTOS which is open source and comes in bundled with its libraries which makes it really easy to use. You can have a better look at free RTOS functions and API by clicking this link. For the previous labs we have been using the HAL library, but from now on we will also be using the CMSIS RTOS V2 library which act as an abstraction layer for free RTOS.

Lab 3 and 4 will be focused on RTOS and some of its important concepts. Lab 3 will cover the following:

- Tasks and Task priorities
- Memory management
- Queues

2. Tasks and Task Priorities

In real time systems, we prefer using different tasks and assigning them with different priorities according to the importance and timing deadlines of that task. Lets say for example an autonomous car's RTOS has 2 functions. First to light up the break lights if the break paddle is pressed and second is to slow the car if it predicts a collusion. Here, the priority should be to slow the car, as delaying this task can be fatal. Figure 1 shows how different tasks with different priorities are executed.

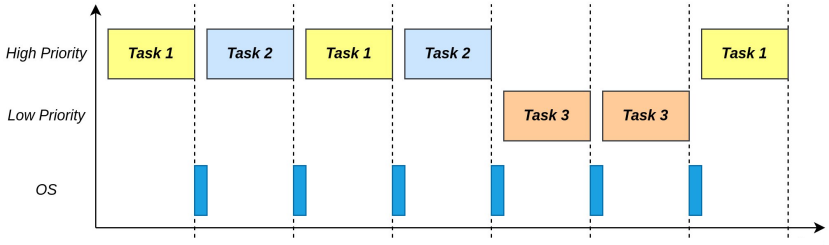


Figure 1: Multiple tasks in RTOS

As it can be seen, each task executes after a certain amount of time, after which a Sys Tick interrupt occurs, halting the current task being executed and initiating or resuming other tasks. This keeps on going and different tasks keep on executing. One thing to be noted is, that after each interrupt, the OS also takes some time. Actually after each task switching, the OS utilises some time for the switching and to perform its own functions.

The shifting of tasks, deciding which task is to be executed and which task is to be paused, is task scheduler's responsibility. Task scheduler is a core component that governs the execution of multiple tasks. It allocates time to tasks based on their priority, ensuring that higher-priority tasks are executed before lower-priority ones. The task scheduler plays a crucial role in meeting real-time requirements by managing task execution in a timely manner. So, every time the Sys Tick interrupt occurs, the task scheduler shifts the task. While using the STM IDE, one problem that might be encountered is that, the HAL API by default, uses the Sys Tick timer to measure how much time has passed since the controller restarted. This is useful for the functions such

as, `HAL_Delay()` or `HAL_GetTick()`. HAL requires this timer to be set at a very high priority in order to keep accurate time, free RTOS on the other hand requires the time base of its' scheduler to be set at a very low priority. An easy work around for this is to use Sys Tick timer for the RTOS and another timer for HAL.

To understand how all this is done in the STM IDE, lets look at an example to create two tasks with different priorities to blink the on-board LED with different speeds. Open the STM IDE and create a new project. In the GUI window, go to the *middleware and software packs*, and select *FREERTOS*. Here, change the interface from disabled to CMSIS_V2. This is shown in Figure 2a.

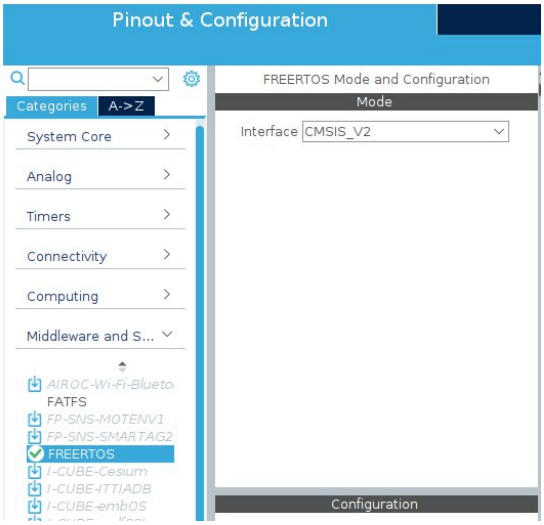


Figure 2a : RTOS Middleware Setup

In the configuration section,you will be able to see different parameters and configurations for RTOS. For the time being, let them be the same. **One thing to be noted in the configurations is that preemption is enabled which enables the scheduler to stop a task and run another higher priority task.** Now, lets create our tasks to blink the LED at different speeds. In the configuration section, select *Tasks and Queues.* You will be able to see a default task here with a priority of *normal priority*. Double click the task and change the name to task_1 and the entry function to start_task_1. Click *OK*. This is shown in Figure 2b. An entry function is a function that is called in order to start the function. Now click the *Add* button and another pop up, similar to the previous one, will appear. Let's name the task as task_2 and its entry function as start_task_2. Keep its priority to be *osPriorityBelowNormal* which will make the task_1 as a higher priority. This is shown in Figure 2c.

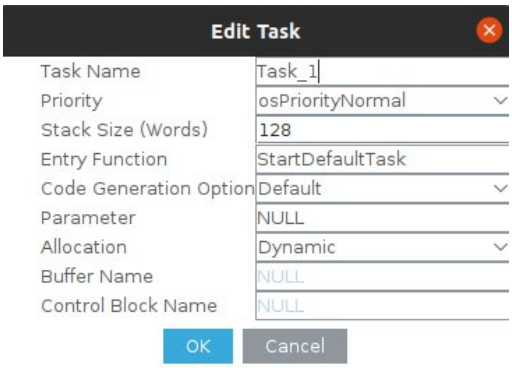


Figure 2b : Task 1

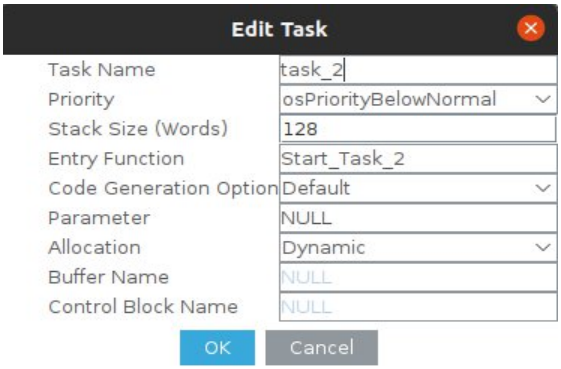


Figure 2c : Task 2

Now to address the problem mentioned about the Sys Tick and changing the timer for the system core, go to the System core in the GUI, select the time base and from the drop down menu, select timer 4 which should resolve the problem.

Now save, and the IDE will generate the codes with FREERTOS middleware as well as the tasks we just made. If you now dive into the main.c file, you will be able to see the entry functions declared by us as well as some OS kernel functions. If you scroll down, you will be able to see the `Start_Task_1` and `Start_Task_2` function templates for our 2 tasks with their own forever loops. Lets write simple led toggle codes in these loops with a delay of 800 ms

and 1000 ms in task 1 and 2 respectively. This is shown in Figure 3.

One thing that can noted is that, to produce a delay, instead of `HAL_Delay()`, `osDelay()` is used. The reason for this is that, the `osDelay()` is an RTOS function and allows another task to run while the delay is occurred in one task, whereas, the `HAL_Delay()` might make the complete system to halt. It is a good practice to use the `osThreadTerminate(NULL)`, at the end of each task which makes the task to terminate gracefully in case something that might break the forever loop to break accidentally.

Now build and run the code on your STM32 and observe how both the tasks compete to toggle the LED.

```
/* USER CODE END Header_Start_Task_1 */
void Start_Task_1(void *argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
      // Toggle LED
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    osDelay(800); // delay 800 ms
  }
  /* USER CODE END 5 */
  // terminate thread in case something happens
  osThreadTerminate(NULL);
}

/* USER CODE BEGIN Header_Start_Task_2 */
/**
* @brief Function implementing the task_2 thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Start_Task_2 */
void Start_Task_2(void *argument)
{
  /* USER CODE BEGIN Start_Task_2 */
  /* Infinite loop */
  for(;;)
  {
      // Toggle LED
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    osDelay(1000); // delay 1 s
  }
  // terminate thread in case something happens
    osThreadTerminate(NULL);
  /* USER CODE END Start_Task_2 */
}
```

Task 1: *Consider the blue button on your board to be a signal indicating the car needs to be slowed, which in return pushes the break. Consider the break to be the on board led. Moreover , when ever the slow down signal is received, the system also shows a message on UART that the car is slowing down. Your task is to create 2 tasks: one for the breaking and the other for displaying the message. Assign them the priority according to the importance of the task. Copy the code with comments, in your report and show the screen shots of your output. Also briefly explain how your code works. Discuss what priority was given to which task and why? (35 marks)*

3. Memory Management

While writing a code especially in RTOS, it is really important to understand where the data in use is stored in memory. This will help us prevent stack overflow and memory leaks which can cause the code to crash or the controller to restart. Lets try to understand where different types of data is stored in the memory of a controller. Consider Figure 4. The section of the memory or the RAM that stores the static and global variables is called called the static memory. The compiler knows how much RAM is to be allocated to this memory at the time of the code compilation and the static memory is not used for any other variables of the program. The local variables that are inside of a function are pushed into the stack memory. The variables are allocated towards the stack when the function is called and unallocated when the function returns. While the compiler will reserve as much stack as it thinks will be needed for local variables, the stack can increase in size automatically allocating. The third part of the memory is called the heap which keeps the dynamic variables or dynamic allocation and it mostly grows towards the stack. This is the section where the programer tells the memory to create space for what the programmer is about to store. It is important to note that in c and c++, it is required by the programmer to free up any memory allocated dynamically. If this is not done, the heap can keep on growing called a memory leak. This could end up mixing up the heap with the stack and can cause the program to crash and cause other undefined behaviors.

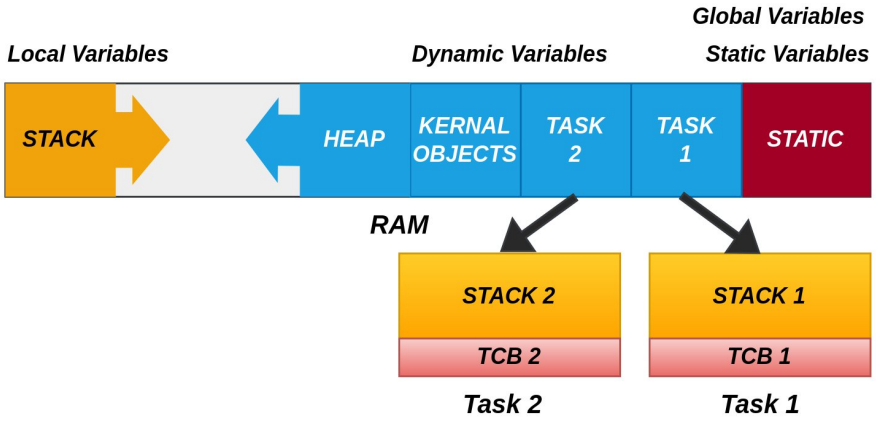


Figure 4 : Memory Management and Allocation

In RTOS, when a new task is created, the task is assigned some memory from the heap. This memory is divided into 2 parts, the task control block (TCB) and stack specifically for that task. The TCB stores information about the position of the task in the heap, task priority level etc. It is defined by the programmer that how much memory is to be assigned for a specific task. If enough memory is not kept aside for a specific task, you might risk causing memory overwriting. Other kernel objects like semaphores, queues and mutexes, about which we will learn about in the upcoming lab, are also stored in the heap. Whenever the memory is dynamically allocated, the program tries to find the largest continuous block of memory and gives that back. If this is done constantly, it might fragment the heap which end up growing the heap towards the stack quickly even if the free slots are available. Hence, for the memory and the code to work properly we want to keep the unused memory free and keep the block of free memory together. To resolve this problem, free RTOS provides with few heap schemes which can be used according to your need. You can have a detailed overview of these from this link. Mostly heap 4 is used by the programmers, but where it is strictly prohibited to use dynamic location, it is preferred to use heap 1. Heaps tasks care of most of the stuff. Lets understand from the example.

Assume you are to write a task that has an array of 200 integers and every element of that array is to be printed. Now open GUI and create a task as mentioned in the previous section. One thing that you will notice in the task is the stack memory. This is the memory that is in the heap and associated to the task. By default it is 128 words or 128x4 = 512 bytes. Now save the task as you did for the previous section and jump into the code. Now when this code is built and run, the controller only prints “starting task” and then hangs. The reason behind this is the the stack size of the task is less than required by the task memory. As 200 integers is equal to 200 x 4 = 800 bytes which exceeds the memory that we have for the task. If we increase the stack memory of the task to 250 then the code will start working again as the memory for the task is now increased. One thing to be noted here is that, heap 4 is scheme is being used here. To use another scheme you can go to *config parameters* of free RTOS. This is shown in Figure 6.

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
    int arr[200]; // integer array to be printed
    char num_1[4]; // string to store the char form of numbers
    char msg[] = "starting Task"; // message
    // printing message
    HAL_UART_Transmit(&huart2,(char*)msg,strlen(msg), HAL_MAX_DELAY);
    osDelay(100); // delay of 100 ms
  /* Infinite loop */
  for(;;)
  {
      // preforming a simple operation on each array and printing
      for (int i=0;i<100;i++)
      {
        arr[i] = 2 + 6;
        sprintf(num_1, " %d\n", arr[i]);
        HAL_UART_Transmit(&huart2,(char*)num_1,strlen(num_1), HAL_MAX_DELAY);
      }
    //osDelay(100);

       osDelay(500); // delay of 500 ms
  }


  osThreadTerminate(NULL); // terminate if something odd happens
  /* USER CODE END 5 */
}
```

Figure 5: Memory Management example

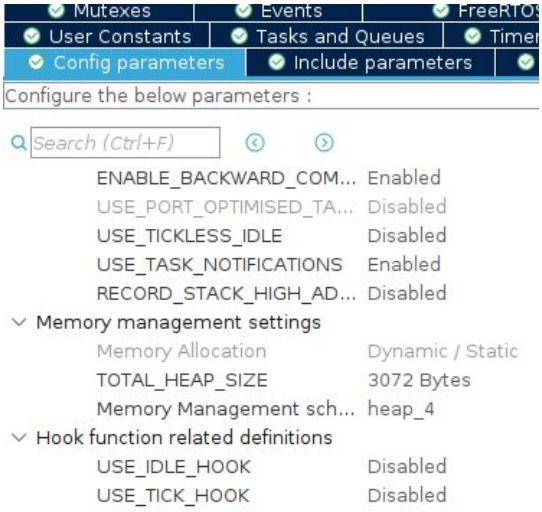


Figure 6: Memory Management Configuration

If you absolutely require some memory block which is greater than the memory that you have for the task, you can use MALOC() and free(), which are thread safe for CMSIS_RTOSV2 in the STM IDE.

4. Queues

While communication between multiple tasks, a global variable can be used to pass data from one variable to another, but this can cause problems. Lets look at an example shown in Figure 7. Consider a global variable that stores 64 bit data. The data is stored in the global variable by Task 1 and Task 2 and retrieved by Task 3. If the data is stored at once then the Task 3 will get the right data, but consider what if the Task 1 writes the first 32 bits and the control then shifts to Task 3. This would cause the Task 3 to have garbage value as the last 32 bits were changed. For another scenario, consider the Task 1 saves all the data in the global variable, but before the Task 3 can retrieve it, Task 2 overwrites the global variable, making the Task 3 receive wrong values. To resolve such problems, we can use Kernel objects like Mutex and Semaphore which we will look in detail in the next lab. Another method to resolve this problem is by the help of queues.

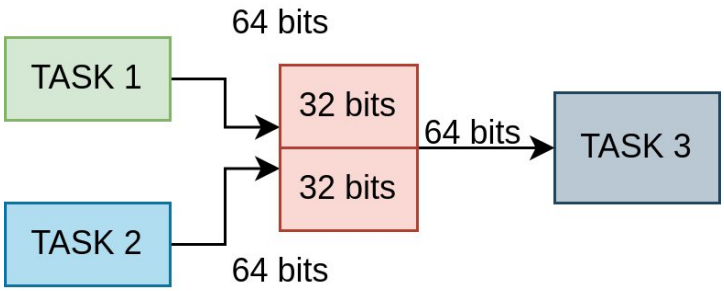


Figure 7: Pitfalls in global variable access

A queue, as the name suggests is a queue of data which treats the data in FIFO manner. Queues usually have global scope so they can be be accessed by multiple tasks. Moreover, the storing of the data in queues is atomic i.e. until the complete data is not stored in the queue, no other task can interrupt it.
Another thing to be noted is that storing the data in queue is done by value and not by reference, meaning that when a variable is copied to a queue, the entire contents of that variable is copied to it. FREERTOS queues are not bound by type so you can save or copy any data type into the queues.

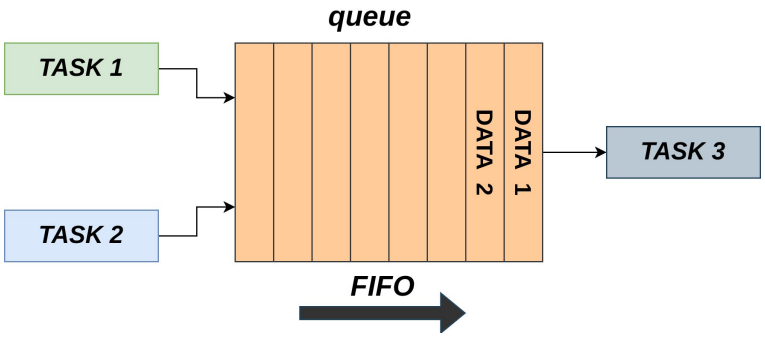


Figure 8: Queues block diagram

As shown in Figure 8, whichever data is sent first in the queue will be received first by the Task 3. If there is no data in the queue, the receive function in the Task 3 will return a false status indicating that the queue is empty. In the same way, if the queue is full, the send functions in Task 1 or Task 2 will return a false status indicating that the queue is full.

Now that we know how queues work and why they are used, lets have a look in STM IDE to see how to set them up and work with them.

Go into the GUI, from middleware, select FreeRTOS and select tasks

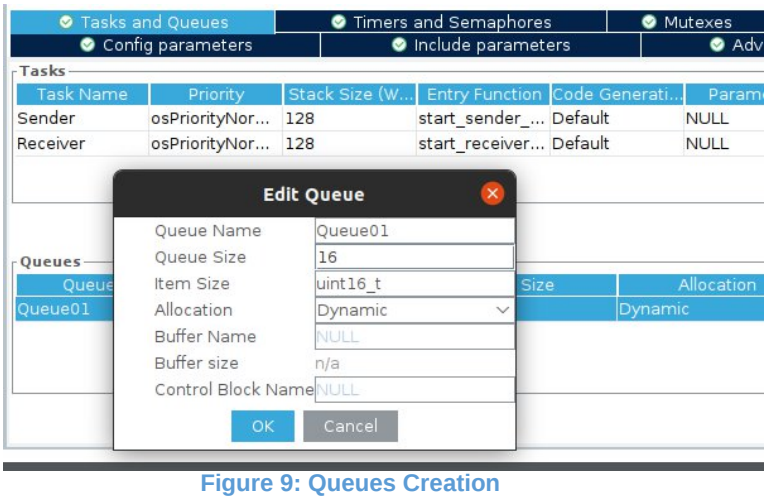


Figure 9: Queues Creation

and queue. Now create 2 tasks, naming receiving and sending as discussed above. Under the tasks section, there is another section named queues. Click add and from there configure a queue as shown in the Figure 9. This will create a queue for you with the attributes mentioned. Now save and the essential code elements will be generated in the main.c file. Lets write a code where one task sends a number, that is incrementing by one, to the queue and the other task prints the data in the queue.

The function `osMessageQueuePut(Queue_Handle,address of the data,priority, wait time)` copies the data from the `address of the data` to the queue pointed by the `queue_handle`. The `priority` in the function describes the priority, the higher the number, the higher the priority and `wait time` indicates the time that should be waited to put some new data if the queue is full before moving forward.

The function `osMessageQueueGet(Queue_Handle,address of the data,priority, wait time)` copies the data from the queue pointed by the `queue_handle` to the `address of the data`. The `priority` in the function describes the priority, the higher the number, the higher the priority and `wait time` indicates the time that should be waited to receive new data if the queue is empty before moving forward.

Now lets see how the above-mentioned code is executed. The sender task sends the number to the queue with handle `Queue01Handle`. This is different for each queue created and is assigned a queue in the main function. The status of the function is stored in the `osStatus_t` type variable which is used to check whether the status is okay or not. If it is not okay, a message is printed indicating that the queue is full.

The receiver task is similar, but is used to receive the data from the queue.

```
/* USER CODE END Header_start_sender_task */
void start_sender_task(void *argument)
{
  /* USER CODE BEGIN 5 */
    uint16_t num = 0; // incrementing number
    char msg[] = "Queue full \n"; // message
  /* Infinite loop */
  for(;;)
  {
      // put the incrementing number in queue and get queue status
      osStatus_t status = osMessageQueuePut(Queue01Handle, &num, 0, 0) ;
      if (status != osOK) // if queue status is full / not okay
      {
          HAL_UART_Transmit(&huart2,(char*)msg,strlen(msg), HAL_MAX_DELAY);
      }
    num = num + 1; // increment by 1
    osDelay(1000); // delay of a second
  }
  /* USER CODE END 5 */
}

/* USER CODE BEGIN Header_start_receiver_task */
/* USER CODE END Header_start_receiver_task */
void start_receiver_task(void *argument)
{
  /* USER CODE BEGIN start_receiver_task */
    uint16_t num; // variable to store the number from queue
    char num_char[5]; // String to print
    char msg[] = "Queue Empty \n"; // message
  /* Infinite loop */
  for(;;)
  {
    // receive the number the queue and get queue status
    osStatus_t status = osMessageQueueGet(Queue01Handle, &num, 0, 0);
    if (status != osOK) // if queue status is empty / not okay
        {
                HAL_UART_Transmit(&huart2,(char*)msg,strlen(msg), HAL_MAX_DELAY);
        }

        sprintf(num_char, "%d\r\n", num);
        HAL_UART_Transmit(&huart2, (char*)num_char, strlen(num_char), HAL_MAX_DELAY);
    osDelay(1000); // delay of a second

  }
  /* USER CODE END start_receiver_task */
}
```

Figure 10: Queues Example code

In the situation of the code just the numbers will be displayed in the UART and now message will be printed. The reason for that is that the delay in both the tasks is same the queue is filled and emptied at the same pace. If the delay in the receiving task is decreased to 500 ms, a message will be displayed after some time indicating that the queue is empty, as it is not being filled with the same speed as it is being emptied. And the opposite will happen if the delay in the send task is decreased to 500 ms.

***Task 2**: Create 2 tasks and 3 queues where the first task should echo the data it receives as it is to the UART. The data received via UART should be through interrupt and should be 11 characters. If the [2-6] characters received by UART are "DELAY" the task 1 should extract the [8-11] characters as integers and send it to queue 1. For example if the data is "-DELAY 1000", the task 1 should put 1000 as uint16_t in queue 1. Task 1 should also print (on UART) the numbers received in queue 2 and 3 whenever they are updated.*

When ever some new data is put in queue 1, Task 2 should toggle the on-board LED with that speed. Whenever the led toggles 10 times, the task 2 should increment a number and put it in queue 2. Moreover whenever queue 1 receives a data, an int array of length 150 should should randomly store integers in each element and put the largest of these random integers into queue 3.

Describe the size and width of each queue and justify. Also describe how were you able to adjust the 150 element int array in the task 2. Copy the code with comments, in your report and show the screen shots of your output. Briefly explain how your code works for each task. (65 marks)